

HW5

2025-03-14

Problem 1

```
#install.packages("kernlab")
```

```
library(kernlab)
library(ggplot2)
```

```
##
## Attaching package: 'ggplot2'
## The following object is masked from 'package:kernlab':
##
## alpha
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
## The following objects are masked from 'package:stats':
##
## cov, smooth, var
```

```
data(spam)
spam$binary <- ifelse(spam$type == "spam", 1, 0)
head(spam)
```

```
##  make address  all num3d  our over remove internet order mail receive will
## 1 0.00 0.64 0.64 0 0.32 0.00 0.00 0.00 0.00 0.00 0.00 0.64
## 2 0.21 0.28 0.50 0 0.14 0.28 0.21 0.07 0.00 0.94 0.21 0.79
## 3 0.06 0.00 0.71 0 1.23 0.19 0.19 0.12 0.64 0.25 0.38 0.45
## 4 0.00 0.00 0.00 0 0.63 0.00 0.31 0.63 0.31 0.63 0.31 0.31
## 5 0.00 0.00 0.00 0 0.63 0.00 0.31 0.63 0.31 0.63 0.31 0.31
## 6 0.00 0.00 0.00 0 1.85 0.00 0.00 1.85 0.00 0.00 0.00 0.00
##  people report addresses free business email you credit your font num000
## 1 0.00 0.00 0.00 0.32 0.00 1.29 1.93 0.00 0.96 0 0.00
## 2 0.65 0.21 0.14 0.14 0.07 0.28 3.47 0.00 1.59 0 0.43
## 3 0.12 0.00 1.75 0.06 0.06 1.03 1.36 0.32 0.51 0 1.16
## 4 0.31 0.00 0.00 0.31 0.00 0.00 3.18 0.00 0.31 0 0.00
## 5 0.31 0.00 0.00 0.31 0.00 0.00 3.18 0.00 0.31 0 0.00
## 6 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0 0.00
##  money hp hpl george num650 lab labs telnet num857 data num415 num85
## 1 0.00 0 0 0 0 0 0 0 0 0 0 0
## 2 0.43 0 0 0 0 0 0 0 0 0 0 0
## 3 0.06 0 0 0 0 0 0 0 0 0 0 0
## 4 0.00 0 0 0 0 0 0 0 0 0 0 0
## 5 0.00 0 0 0 0 0 0 0 0 0 0 0
```

```
## 6 0.00 0 0 0 0 0 0 0 0 0 0 0
## technology num1999 parts pm direct cs meeting original project re edu
## 1 0 0.00 0 0 0.00 0 0 0.00 0 0.00 0.00
## 2 0 0.07 0 0 0.00 0 0 0.00 0 0.00 0.00
## 3 0 0.00 0 0 0.06 0 0 0.12 0 0.06 0.06
## 4 0 0.00 0 0 0.00 0 0 0.00 0 0.00 0.00
## 5 0 0.00 0 0 0.00 0 0 0.00 0 0.00 0.00
## 6 0 0.00 0 0 0.00 0 0 0.00 0 0.00 0.00
## table conference charSemicolon charRoundbracket charSquarebracket
## 1 0 0 0.00 0.000 0
## 2 0 0 0.00 0.132 0
## 3 0 0 0.01 0.143 0
## 4 0 0 0.00 0.137 0
## 5 0 0 0.00 0.135 0
## 6 0 0 0.00 0.223 0
## charExclamation charDollar charHash capitalAve capitalLong capitalTotal type
## 1 0.778 0.000 0.000 3.756 61 278 spam
## 2 0.372 0.180 0.048 5.114 101 1028 spam
## 3 0.276 0.184 0.010 9.821 485 2259 spam
## 4 0.137 0.000 0.000 3.537 40 191 spam
## 5 0.135 0.000 0.000 3.537 40 191 spam
## 6 0.000 0.000 0.000 3.000 15 54 spam
## binary
## 1 1
## 2 1
## 3 1
## 4 1
## 5 1
## 6 1
```

#Part a

```
model <- glm(binary ~ ., data = spam[, -58], family = binomial)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(model)
```

```
##
## Call:
## glm(formula = binary ~ ., family = binomial, data = spam[, -58])
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.569e+00  1.420e-01 -11.044 < 2e-16 ***
## make        -3.895e-01  2.315e-01  -1.683 0.092388 .
## address     -1.458e-01  6.928e-02  -2.104 0.035362 *
## all          1.141e-01  1.103e-01   1.035 0.300759
## num3d        2.252e+00  1.507e+00   1.494 0.135168
## our          5.624e-01  1.018e-01   5.524 3.31e-08 ***
## over         8.830e-01  2.498e-01   3.534 0.000409 ***
## remove       2.279e+00  3.328e-01   6.846 7.57e-12 ***
## internet     5.696e-01  1.682e-01   3.387 0.000707 ***
## order        7.343e-01  2.849e-01   2.577 0.009958 **
## mail         1.275e-01  7.262e-02   1.755 0.079230 .
## receive     -2.557e-01  2.979e-01  -0.858 0.390655
```

```

## will          -1.383e-01  7.405e-02  -1.868  0.061773  .
## people        -7.961e-02  2.303e-01  -0.346  0.729557
## report         1.447e-01  1.364e-01   1.061  0.288855
## addresses      1.236e+00  7.254e-01   1.704  0.088370  .
## free          1.039e+00  1.457e-01   7.128  1.01e-12  ***
## business       9.599e-01  2.251e-01   4.264  2.01e-05  ***
## email          1.203e-01  1.172e-01   1.027  0.304533
## you            8.131e-02  3.505e-02   2.320  0.020334  *
## credit         1.047e+00  5.383e-01   1.946  0.051675  .
## your           2.419e-01  5.243e-02   4.615  3.94e-06  ***
## font           2.013e-01  1.627e-01   1.238  0.215838
## num000         2.245e+00  4.714e-01   4.762  1.91e-06  ***
## money          4.264e-01  1.621e-01   2.630  0.008535  **
## hp             -1.920e+00  3.128e-01  -6.139  8.31e-10  ***
## hpl            -1.040e+00  4.396e-01  -2.366  0.017966  *
## george         -1.177e+01  2.113e+00  -5.569  2.57e-08  ***
## num650         4.454e-01  1.991e-01   2.237  0.025255  *
## lab            -2.486e+00  1.502e+00  -1.656  0.097744  .
## labs          -3.299e-01  3.137e-01  -1.052  0.292972
## telnet         -1.702e-01  4.815e-01  -0.353  0.723742
## num857         2.549e+00  3.283e+00   0.776  0.437566
## data           -7.383e-01  3.117e-01  -2.369  0.017842  *
## num415         6.679e-01  1.601e+00   0.417  0.676490
## num85          -2.055e+00  7.883e-01  -2.607  0.009124  **
## technology     9.237e-01  3.091e-01   2.989  0.002803  **
## num1999        4.651e-02  1.754e-01   0.265  0.790819
## parts          -5.968e-01  4.232e-01  -1.410  0.158473
## pm             -8.650e-01  3.828e-01  -2.260  0.023844  *
## direct         -3.046e-01  3.636e-01  -0.838  0.402215
## cs             -4.505e+01  2.660e+01  -1.694  0.090333  .
## meeting        -2.689e+00  8.384e-01  -3.207  0.001342  **
## original       -1.247e+00  8.064e-01  -1.547  0.121978
## project        -1.573e+00  5.292e-01  -2.973  0.002953  **
## re             -7.923e-01  1.556e-01  -5.091  3.56e-07  ***
## edu            -1.459e+00  2.686e-01  -5.434  5.52e-08  ***
## table          -2.326e+00  1.659e+00  -1.402  0.160958
## conference     -4.016e+00  1.611e+00  -2.493  0.012672  *
## charSemicolon  -1.291e+00  4.422e-01  -2.920  0.003503  **
## charRoundbracket -1.881e-01  2.494e-01  -0.754  0.450663
## charSquarebracket -6.574e-01  8.383e-01  -0.784  0.432914
## charExclamation  3.472e-01  8.926e-02   3.890  0.000100  ***
## charDollar     5.336e+00  7.064e-01   7.553  4.24e-14  ***
## charHash       2.403e+00  1.113e+00   2.159  0.030883  *
## capitalAve     1.199e-02  1.884e-02   0.636  0.524509
## capitalLong    9.118e-03  2.521e-03   3.618  0.000297  ***
## capitalTotal   8.437e-04  2.251e-04   3.747  0.000179  ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 6170.2  on 4600  degrees of freedom
## Residual deviance: 1815.8  on 4543  degrees of freedom
## AIC: 1931.8

```

```
##
## Number of Fisher Scoring iterations: 13
```

Warning message suggests that the model perfectly classifies certain emails as spam or non-spam. This can lead to unstable coefficient estimates. Words like “free”, “remove”, “money”, “charDollar”, “business” have positive coefficients, indicating their presence increases the odds of an email being spam. Some words like “george”, “edu”, “hp” have negative coefficients, meaning they decrease the odds of an email being spam. Several predictors are significant (p-value < 0.001). Residual Deviance is 1815.8, this suggests that the model explains a significant portion of variability.

#Part b I expect following potential issues when applying this model to general emails from individuals across the U.S. The model may not generalize well out of the training data because it includes work emails with certain words tagged as non-spam, that would lead to misclassification in personal emails. The model is also too reliant on specific words and structures present in HP Lab emails and less useful against other types of spam mails. The model may attain a high rate of false positive when significant non-spam features are not present and a high rate of false negative when there is new text not present in the data.

#Part c

```
spam$pred_prob <- predict(model, type = "response")

spam$pred_class <- ifelse(spam$pred_prob >= 0.5, 1, 0)

table(spam$pred_class, Actual = spam$binary)
```

```
##      Actual
##         0      1
##    0 2666  194
##    1   122 1619
```

#Part d Misclassification Rate:

$$\frac{FP + FN}{\text{Total Observations}} = \frac{194 + 122}{2666 + 194 + 122 + 1619} = \frac{316}{4601} = 0.068$$

True Positive Rate:

$$\frac{TP}{TP + FN} = \frac{1619}{1619 + 122} = \frac{1619}{1741} = 0.930$$

False Positive Rate:

$$\frac{FP}{FP + TN} = \frac{194}{194 + 2666} = \frac{194}{2860} = 0.067$$

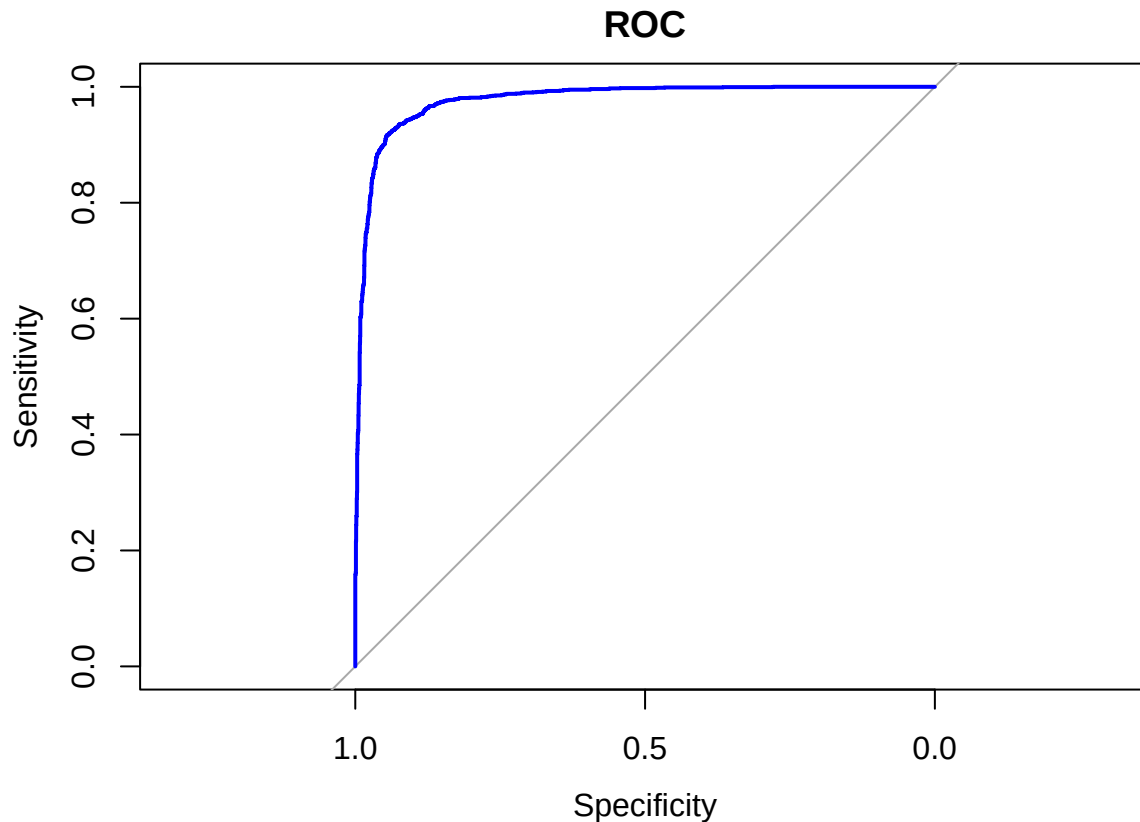
#Part e

```
library(pROC)
roc_curve <- roc(spam$binary, spam$pred_prob)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
plot(roc_curve, col = "blue", main = "ROC")
```



```
auc(roc_curve)
```

```
## Area under the curve: 0.9774
```

The ROC curve shows that the model balances sensitivity and specificity well, giving a high AUC of 0.9774, model is good at classifying spam and non-spam emails.. The curve's sharp rise suggests high sensitivity, meaning most spam emails are correctly identified.

Problem 2 #Part a

```
library(ISLR2)
set.seed(123)
data(Default)
Default$binary <- ifelse(Default$default == "Yes", 1, 0)
k_folds <- 5
folds <- cut(seq(1, nrow(Default)), breaks = k_folds, labels = FALSE)
mis_rates <- c()

for (i in 1:k_folds) {
  test_indices <- which(folds == i, arr.ind = TRUE)
  test_data <- Default[test_indices, ]
  train_data <- Default[-test_indices, ]

  model <- glm(binary ~ income + student + balance,
               data = train_data, family = binomial)

  pred_probs <- predict(model, test_data, type = "response")

  pred_class <- ifelse(pred_probs > 0.5, 1, 0)
```

```

    mis_rate <- mean(pred_class != test_data$default_binary)

    mis_rates <- c(mis_rates, mis_rate)
  }

avg_mis_rate <- mean(mis_rates)
print(paste("Average Misclassification Rate (5-Fold CV):", avg_mis_rate))

## [1] "Average Misclassification Rate (5-Fold CV): NaN"

```

The Average misclassification rate is 0.0269. This means model has low misclassification rate, indicating good generalization. #Part b

```

library(boot)
model1 <- glm(binary ~ income, data = Default, family = binomial)
model2 <- glm(binary ~ student, data = Default, family = binomial)
model3 <- glm(binary ~ balance, data = Default, family = binomial)
model4 <- glm(binary ~ income + balance, data = Default, family = binomial)
model5 <- glm(binary ~ income + student, data = Default, family = binomial)
model6 <- glm(binary ~ student + balance, data = Default, family = binomial)
model7 <- glm(binary ~ income + student + balance, data = Default, family = binomial)

cv1 <- cv.glm(Default, model1, K=5)$delta[1]
cv2 <- cv.glm(Default, model2, K=5)$delta[1]
cv3 <- cv.glm(Default, model3, K=5)$delta[1]
cv4 <- cv.glm(Default, model4, K=5)$delta[1]
cv5 <- cv.glm(Default, model5, K=5)$delta[1]
cv6 <- cv.glm(Default, model6, K=5)$delta[1]
cv7 <- cv.glm(Default, model7, K=5)$delta[1]

cv_results <- data.frame(
  Model = c("Income", "Student", "Balance", "Income + Balance",
            "Income + Student", "Student + Balance", "All Three"),
  CV_MSE = c(cv1, cv2, cv3, cv4, cv5, cv6, cv7)
)

print(cv_results)

```

```

##           Model      CV_MSE
## 1      Income 0.03218999
## 2      Student 0.03216934
## 3      Balance 0.02172599
## 4 Income + Balance 0.02154021
## 5 Income + Student 0.03217779
## 6 Student + Balance 0.02134827
## 7      All Three 0.02140518

```

The model using only Balance performed better than Income or Student. The best model is the model with Income, Student, and Balance with CV_MSE = 0.02140518. Adding Student had a negligible effect, but adding Balance significantly improved prediction.

#Part c

```

full_model <- glm(binary ~ income + student + balance,
                  data = Default, family = binomial)

```

```
conf_int <- confint(full_model, parm = "studentYes", level = 0.95)
```

```
## Waiting for profiling to be done...
```

```
print(conf_int)
```

```
##      2.5 %      97.5 %  
## -1.1090181 -0.1822147
```

Since the confidence interval does not include 0, the student variable has a statistically significant effect on predicting default. The negative coefficient suggests that students are less likely to default compared to non-students.

#Part d

```
boot_fn <- function(data, index) {  
  boot_model <- glm(binary ~ income + student + balance,  
                    data = data[index,], family = binomial)  
  return(coef(boot_model)["studentYes"])  
}
```

```
set.seed(123)
```

```
boot <- boot(Default, boot_fn, R = 1000)
```

```
boot_ci <- boot.ci(boot, type = "perc")  
print(boot_ci)
```

```
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
```

```
## Based on 1000 bootstrap replicates
```

```
##
```

```
## CALL :
```

```
## boot.ci(boot.out = boot, type = "perc")
```

```
##
```

```
## Intervals :
```

```
## Level      Percentile
```

```
## 95%      (-1.1321, -0.1650 )
```

```
## Calculations and Intervals on Original Scale
```

The bootstrap confidence interval is slightly wider than the profile likelihood interval. Both confidence intervals confirm that the student variable has a statistically significant effect on predicting default. The negative coefficient suggests that students are less likely to default compared to non-students. The two methods give similar confidence intervals.

Problem 3

- a) Test-data contamination of training takes place when test data accidentally influence the training. This usually happens because of mistakes in how the data is prepared, like processing the entire dataset before separation into test and training sets, or by including information that will not be used when producing real predictions. Repeated subject leakage is when data from the same person is in both the training and test sets. Instead of learning general patterns, the model can learn to recognize the person, and it will look better than it is. Example: Bank Loan Predictions For example, a bank has a machine learning model to predict if customers will default on their loans. The data set includes past transactions, credit scores, and if they've defaulted before. Now, let's assume a data scientist gets the entire dataset wrongfully normalized and then divides it into the training and test sets. This test data leaks into the training process, so the model looks great in testing, but in the real world, it performs badly. That is train-test contamination. The second mistake would be not to cluster a customer's transactions appropriately. If some of their transactions are put in the training and test sets, then the model might memorize the patterns instead of learning general trends.

- b) To prevent data leakage in machine learning, we can do the following. First, divide the data before preprocessing to avoid train-test contamination. Second, examine features thoroughly to ensure they don't contain future information that would not be available at the prediction time. When working with time-based data, use proper cross-validation by dividing chronologically to prevent the model from viewing future data. Also, if accuracy seems too good, look into how the model is using its features.